

Système d'Irrigation Intelligente sous Changements Climatiques

DSSAT + AI: Projet MLOps End-to-End

Table des Matières

1. Contexte et Problématique
 2. Vue d'ensemble du Projet
 3. Architecture MLOps Complète
 4. Stack Technique Détaillé
 5. Implémentation Phase par Phase
 6. Diagrammes de Workflow
 7. Déploiement et Production
-

Contexte et Problématique

Le Défi

Le Maroc (et le monde) fait face à:

- **Stress hydrique croissant:** Réduction des précipitations de 20-30% d'ici 2050
- **Variabilité climatique:** Températures extrêmes, sécheresses prolongées
- **Inefficacité de l'irrigation:** 60-70% de l'eau agricole gaspillée
- **Baisse des rendements:** Impact sur la sécurité alimentaire

La Solution

Un système intelligent qui combine:

1. **DSSAT (Decision Support System for Agrotechnology Transfer):** Modèle de simulation de croissance des cultures
2. **Machine Learning/Deep Learning:** Prédictions en temps réel et optimisation
3. **MLOps:** Pipeline automatisé de la donnée à la décision

Impact Attendu

-  Réduction de 30-40% de la consommation d'eau
-  Augmentation de 15-25% des rendements
-  Adaptation proactive aux changements climatiques

- ✔ Décisions basées sur les données en temps réel

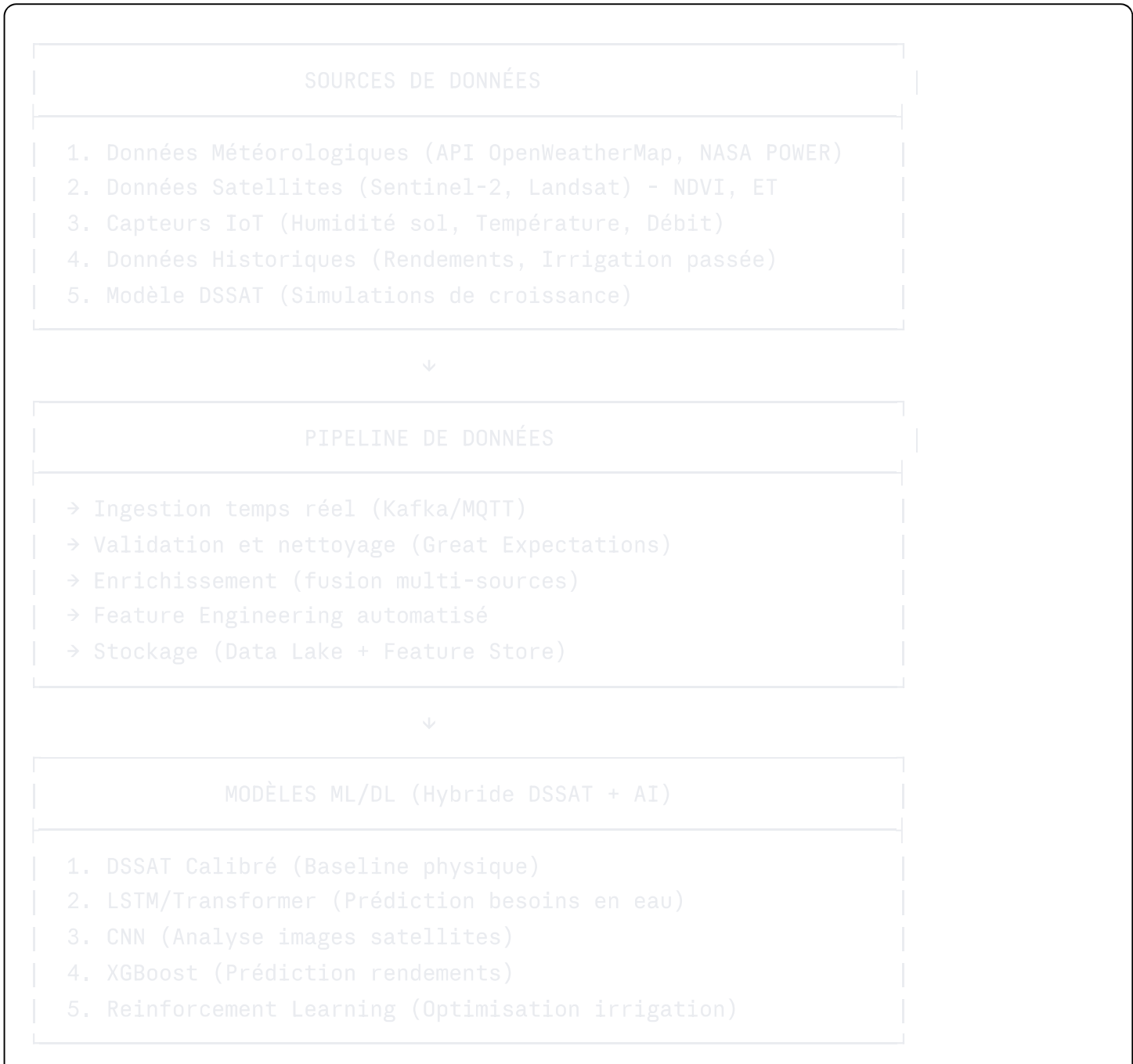
🏗️ Vue d'ensemble du Projet

Objectif Principal

Créer un système de recommandation d'irrigation intelligent qui:

- Prédit les besoins en eau des cultures en temps réel
- S'adapte automatiquement aux prévisions climatiques
- Optimise les calendriers d'irrigation
- Alerte sur les stress hydriques et thermiques
- Fournit des recommandations personnalisées par parcelle

Composantes Clés





SYSTÈME DE DÉCISION

- Recommandations d'irrigation (quand, combien)
- Alertes (stress, maladies, gel)
- Optimisation multi-objectifs (eau, rendement, coût)
- Tableau de bord agriculteur

Architecture MLOps Complète

Architecture Globale

COUCHE D'INGESTION

Météo API
(Temps réel)

- Temp
- Précip.
- Vent
- Radiation

Satellites
(Sentinel-2)

- NDVI
- EVI
- LAI
- ET

IoT Sensors
(MQTT/LoRaWAN)

- Humidité sol
- Température
- Salinité
- Débit eau

KAFKA CLUSTER (Event Streaming)

Topics: weather-data, satellite-data, iot-sensors, irrigation-events

APACHE FLINK (Stream Processing)

- Data Validation (Great Expectations)
- Real-time Aggregation (Fenêtres temporelles)
- Anomaly Detection (Isolation Forest)

- Data Enrichment (Jointure multi-sources)



DATA LAKE (MinIO/S3)

/raw/

/weather/YYYY/MM/DD/

/satellite/YYYY/MM/DD/

/iot/YYYY/MM/DD/

/processed/

/features/

TIMESERIES DB

(TimescaleDB/InfluxDB)

- Mesures temps réel

- Agrégations

- Requêtes rapides

ORCHESTRATION (Apache Airflow / Prefect)

DAG 1: SATELLITE DATA PIPELINE (Daily)

- └ Download Sentinel-2 images

- └ Cloud masking & preprocessing

- └ Calculate NDVI, EVI, LAI

- └ Calculate Evapotranspiration (ET)

- └ Store in Feature Store

DAG 2: DSSAT SIMULATION PIPELINE (Weekly)

- └ Calibrate DSSAT with recent observations

- └ Run crop growth simulations

- └ Generate water stress indicators

- └ Predict crop stages & water needs

- └ Store simulation outputs

DAG 3: FEATURE ENGINEERING PIPELINE (Hourly)

- └ Extract temporal features (7d, 14d, 30d windows)

- └ Calculate crop coefficient (Kc)

- └ Compute water balance

- └ Generate lag features

- └ Update Feature Store

DAG 4: MODEL TRAINING PIPELINE (Weekly)

- └ Load training data from Feature Store

- └ Train ensemble models (LSTM + XGBoost)

- └ Hyperparameter tuning (Optuna)

- └ Evaluate on validation set

- └─ Compare with DSSAT baseline
- └─ Track experiments (MLflow)
- └─ Register best model

DAG 5: MODEL DEPLOYMENT PIPELINE (On-demand)

- └─ Load champion model from registry
- └─ Run A/B testing framework
- └─ Deploy to staging
- └─ Integration tests
- └─ Gradual rollout to production
- └─ Monitor performance metrics

|



FEATURE STORE (Feast)

OFFLINE STORE (PostgreSQL): Historical features pour training
ONLINE STORE (Redis): Real-time features pour inference

Feature Groups:

- weather_features (temp, precip, wind, radiation)
- soil_features (moisture, temperature, EC)
- crop_features (NDVI, LAI, growth_stage, Kc)
- irrigation_history (past_7d_amount, frequency)
- climate_projections (next_7d_forecast)

|



ML/DL MODELS TRAINING

MODEL 1: HYBRID DSSAT-LSTM

DSSAT Physical Model

- └─ Crop growth simulation
- └─ Soil water balance
- └─ Stress indices



LSTM Correction Layer

- └─ Learn DSSAT errors
- └─ Adapt to local conditions
- └─ Improve short-term predictions

MODEL 2: MULTIVARIATE TIME SERIES (Transformer)

Input: [Weather, Soil, Crop, Irrigation] × Time

↓

Temporal Fusion Transformer (TFT)

└─ Multi-horizon forecasting (1-14 days)

└─ Attention mechanisms

└─ Interpretable predictions

↓

Output: [Water_need, Stress_prob, Optimal_timing]

MODEL 3: SATELLITE IMAGE ANALYSIS (CNN)

U-Net for Crop Health Segmentation

└─ Input: Sentinel-2 multispectral (10 bands)

└─ Output: Stress zones, Health map

└─ Transfer learning (ImageNet → AgriNet)

MODEL 4: YIELD PREDICTION (Ensemble)

XGBoost + LightGBM + CatBoost

└─ Features: Weather, Irrigation, NDVI, Growth stage

└─ Stacking ensemble

└─ SHAP for feature importance

MODEL 5: IRRIGATION OPTIMIZATION (RL)

Deep Q-Network (DQN) / PPO

└─ State: Soil moisture, Weather forecast, Crop stage

└─ Action: Irrigation amount & timing

└─ Reward: $\text{Max}(\text{Yield}) - \lambda_1 \times \text{Water_used} - \lambda_2 \times \text{Stress}$

└─ Training: Simulation environment (DSSAT)

EXPERIMENT TRACKING: MLflow

MODEL VERSIONING: DVC + MLflow Model Registry

HYPERPARAMETER TUNING: Optuna with Bayesian Optimization



MODEL SERVING LAYER

FASTAPI SERVICE (Multiple Endpoints)

POST /predict/irrigation

└ Input: field_id, current_date

└ Output: {amount_mm, timing, confidence}

└ Latency: <500ms

POST /predict/yield

└ Input: field_id, growth_stage

└ Output: {predicted_yield_t/ha, uncertainty}

└ Latency: <300ms

POST /analyze/satellite

└ Input: field_polygon, date_range

└ Output: {ndvi_map, stress_zones, recommendations}

└ Latency: <2s

GET /forecast/climate

└ Input: location, horizon_days

└ Output: {temp, precip, et0, confidence_intervals}

└ Latency: <200ms

CACHING LAYER: Redis

└ Cache weather forecasts (TTL: 6h)

└ Cache model predictions (TTL: 1h)

└ Cache satellite indices (TTL: 24h)

MODEL SERVER: Triton Inference Server / TorchServe

└ GPU acceleration for CNN models

└ Batch inference optimization

└ Model versioning & A/B testing



MONITORING & OBSERVABILITY

DATA MONITORING (Evidently AI)

└ Data drift detection (PSI, KS test)

└ Feature distribution monitoring

└ Missing data alerts

└ Data quality reports

MODEL MONITORING (WhyLabs / Arize)

- └ Prediction drift (compared to training distribution)
- └ Model performance degradation
- └ Accuracy vs DSSAT baseline
- └ Confidence calibration

SYSTEM MONITORING (Prometheus + Grafana)

- └ API latency & throughput
- └ Model inference time
- └ Resource utilization (CPU, GPU, Memory)
- └ Error rates & success rates

BUSINESS METRICS TRACKING

- └ Water savings (m³/ha)
- └ Yield improvements (%)
- └ Farmer satisfaction (NPS)
- └ ROI calculation

ALERTING SYSTEM

- └ Slack/Email notifications
- └ SMS for critical irrigation alerts
- └ Dashboard alerts (Grafana)
- └ PagerDuty for system failures



USER INTERFACES

MOBILE APP (React Native / Flutter)

For Farmers:

- └ Daily irrigation recommendations
- └ Weather forecasts
- └ Crop health alerts
- └ Satellite imagery visualization
- └ Irrigation history & analytics

WEB DASHBOARD (React + Plotly.js)

For Agronomists & Managers:

- └ Multi-field overview
- └ Performance analytics
- └ DSSAT simulation results

Stream Processing:

- **Apache Kafka**: Event streaming
- **Apache Flink**: Real-time processing
- **MQTT**: IoT sensor communication

Batch Processing:

- **Apache Spark**: Large-scale data processing
- **Dask**: Python-native distributed computing

Storage:

- **MinIO**: S3-compatible object storage
- **PostgreSQL**: Relational data
- **TimescaleDB**: Time-series optimization
- **InfluxDB**: Alternative time-series DB

Workflow Orchestration:

- **Apache Airflow**: Primary choice
- **Prefect**: Modern alternative
- **Dagster**: Data-aware orchestration

Machine Learning & AI

yaml

Deep Learning Frameworks:

- PyTorch: Primary framework
- PyTorch Lightning: Training abstraction
- TensorFlow: For specific models

Classical ML:

- Scikit-learn: Baseline models
- XGBoost/LightGBM/CatBoost: Gradient boosting

Time Series:

- Prophet: Facebook's forecasting tool
- ARIMA/SARIMA: Statistical baselines
- Temporal Fusion Transformer: SOTA for multivariate

Reinforcement Learning:

- Stable-Baselines3: RL algorithms
- Ray RLlib: Distributed RL

Computer Vision:

- OpenCV: Image processing
- Rasterio/GDAL: Geospatial data
- Segmentation Models PyTorch: U-Net, DeepLab

Optimization:

- Optuna: Hyperparameter tuning
- Ray Tune: Distributed HPO

MLOps Tools

yaml

Experiment Tracking:

- **MLflow**: Experiments, models, registry
- **Weights & Biases**: Alternative

Data Versioning:

- **DVC**: Data version control
- **Pachyderm**: Data lineage

Feature Store:

- **Feast**: Open-source feature store
- **Tecton**: Enterprise alternative

Model Serving:

- **FastAPI**: REST API
- **Triton Inference Server**: NVIDIA GPU optimization
- **TorchServe**: PyTorch native serving
- **BentoML**: Model packaging

Monitoring:

- **Prometheus**: Metrics collection
- **Grafana**: Visualization
- **Evidently AI**: ML monitoring
- **WhyLabs**: Production ML observability
- **Arize**: Model performance monitoring

Agricultural Modeling

yaml

Crop Modeling:

- **DSSAT**: v4.8+ (Fortran/Python interface)
- **APSIM**: Alternative crop model
- **AquaCrop**: FAO water productivity model

Geospatial:

- **Google Earth Engine**: Satellite data
- **Sentinel Hub**: API for Sentinel data
- **Planetary Computer**: Microsoft's geospatial data

Weather Data:

- **OpenWeatherMap API**: Current & forecast
- **NASA POWER**: Historical & forecast
- **ECMWF**: High-quality forecasts
- **Météo Maroc**: Local data

Frontend & Visualization

yaml

Web:

- React: Modern UI framework
- Next.js: SSR React framework
- Plotly.js: Interactive plots
- Leaflet: Map visualization

Mobile:

- React Native: Cross-platform
- Flutter: Alternative

Analytics:

- Streamlit: Rapid prototyping
- Dash: Python dashboards
- Grafana: System monitoring

Workflow Détaillé

WORKFLOW 1: Data Ingestion & Processing



▼

```
Storage Layer
├─ Raw: S3/MinIO bucket
│   └─ /raw/weather/2024/05/16/00/
├─ Processed: TimescaleDB
│   └─ Table: weather_hourly
└─ Features: Redis (online)
    └─ Key: weather:latest:{field_id}
```

[Every 4 hours] IoT Sensor Data



```
MQTT Broker → Kafka Bridge
├─ Sensors: Soil moisture (0-100%)
├─ Soil temp (°C)
├─ Soil EC (dS/m)
└─ Water flow meters (L/min)
```



```
Flink: Real-time Aggregation
├─ Window: 1-hour tumbling
├─ Metrics: avg, min, max, stddev
├─ Alert: If soil_moisture < 30%
└─ Write to TimescaleDB + Redis
```

[Daily 06:00 AM] Satellite Data



```
Airflow DAG: satellite_ingestion
├─ Query Sentinel Hub API
├─ Filter: Cloud cover < 20%
├─ Download: 10m resolution
│   └─ Bands: B2,B3,B4,B5,B6,B7,B8,B8A
├─ Save: GeoTIFF → S3
└─ Trigger: satellite_processing DAG
```



```
Satellite Processing DAG
├─ Atmospheric correction (Sen2Cor)
├─ Cloud masking (FMask)
├─ Calculate indices:
│   └─ • NDVI = (NIR-Red)/(NIR+Red)
```

- EVI = Enhanced Vegetation
- LAI = Leaf Area Index
- ET = Evapotranspiration
- └ Zonal statistics per field
- └ Store in PostGIS + Feature Store

WORKFLOW 2: DSSAT Integration & Simulation

WEEKLY DSSAT CALIBRATION & SIMULATION

[Sunday 02:00 AM] DSSAT Pipeline Start



Task 1: Prepare Input Files

- └ Weather file (.WTH)
 - └ Format: DSSAT standard
- └ Soil file (.SOL)
 - └ From soil database
- └ Crop management (.MZX)
 - └ Planting date, variety
- └ Experimental data (.MZA)
 - └ Observed growth, yield



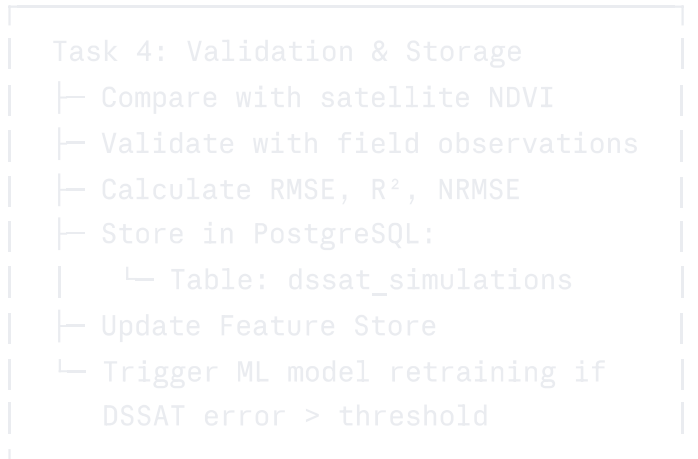
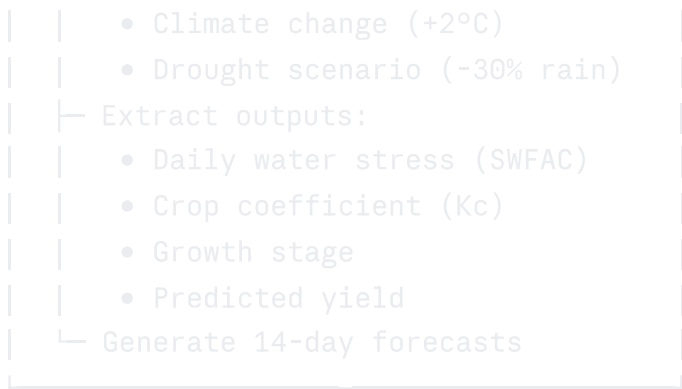
Task 2: DSSAT Calibration

- └ Run GLUE algorithm
 - └ Calibrate cultivar params
- └ Compare simulated vs observed:
 - Biomass
 - Yield
 - LAI
- └ Optimize parameters using Optuna
 - └ Save calibrated .CUL file



Task 3: Seasonal Simulation

- └ Load calibrated parameters
- └ Run simulations for all fields
- └ Scenarios:
 - Current irrigation schedule
 - Optimized irrigation

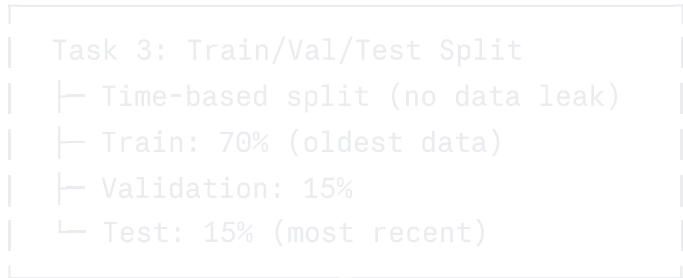
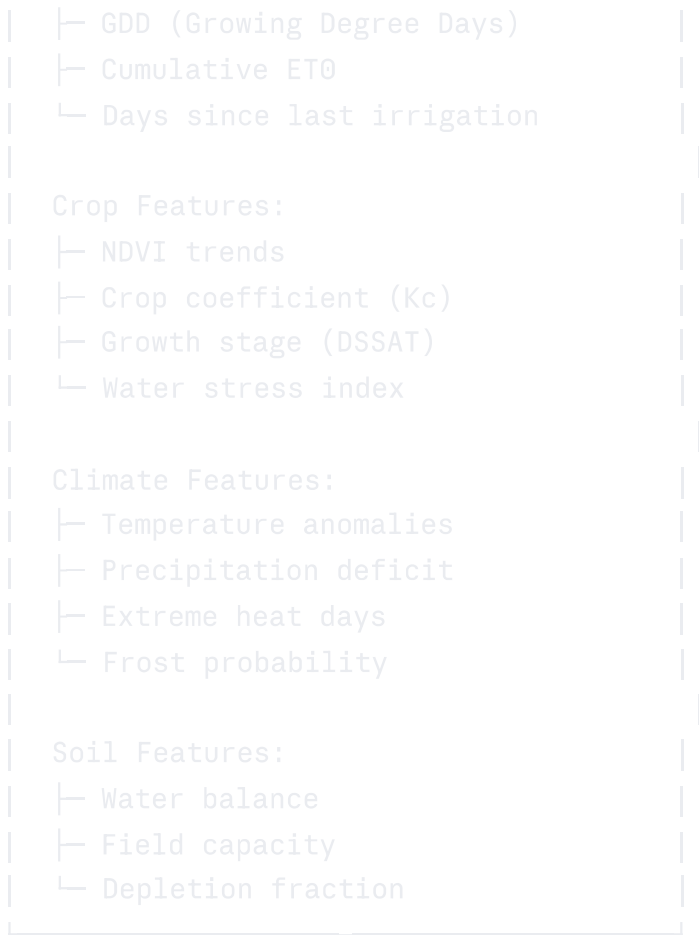


WORKFLOW 3: ML Model Training & Deployment



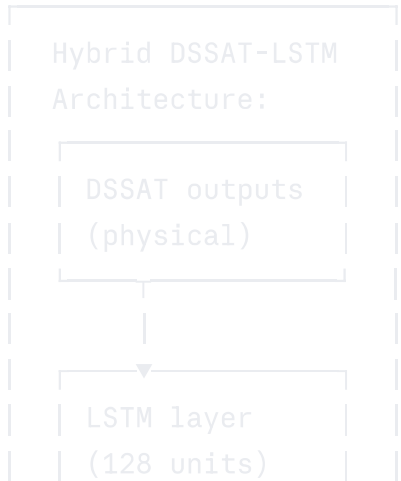
[Monday 01:00 AM] Feature Preparation



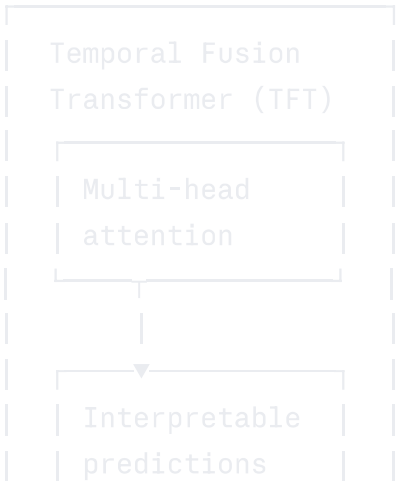


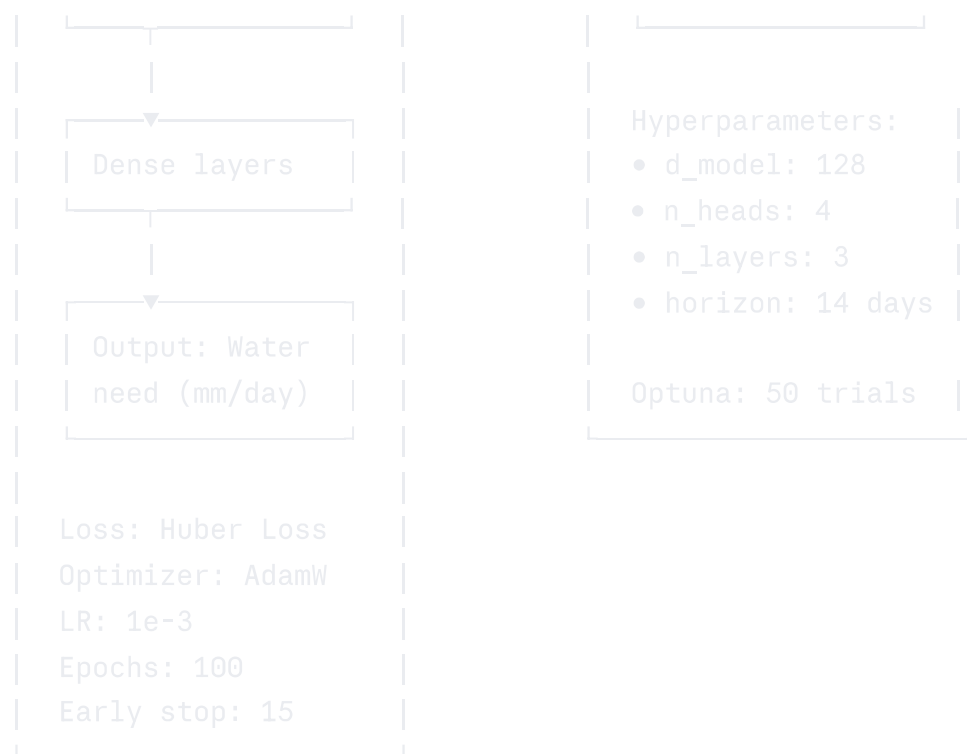
PARALLEL MODEL TRAINING

Branch 1: LSTM Model



Branch 2: Transformer

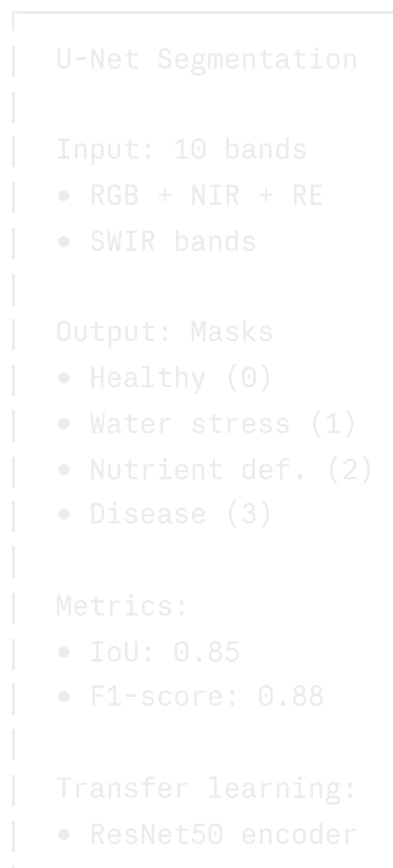




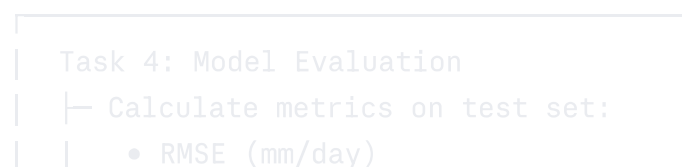
Branch 3: XGBoost

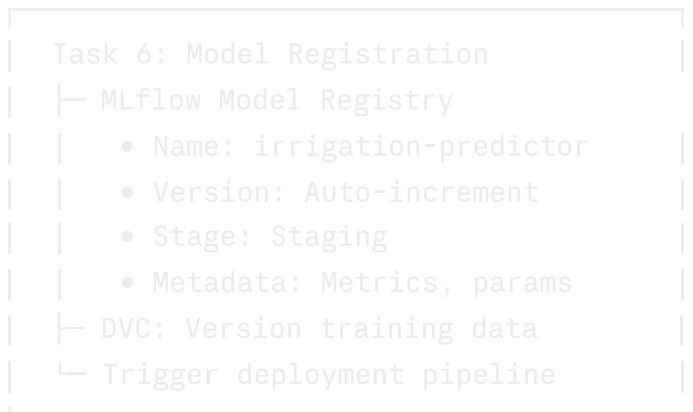
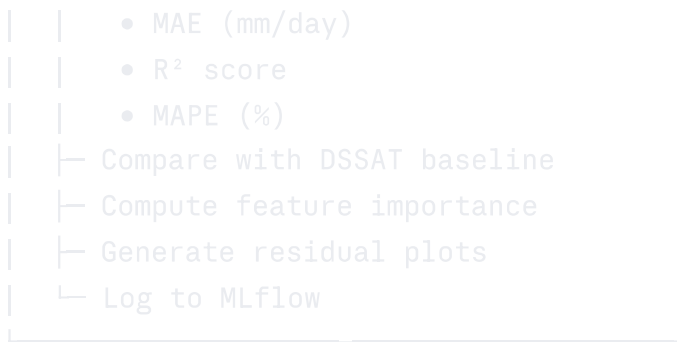


Branch 4: CNN (Satellite)



All models logged to MLflow





WORKFLOW 4: Model Deployment & Serving



[Triggered by model registration]



```
Dockerfile:
FROM pytorch/pytorch:2.1.0-cuda11.8
COPY model/ /app/model/
COPY src/ /app/src/
RUN pip install -r requirements.txt
EXPOSE 8000
CMD ["uvicorn", "main:app"]
```

```
└─ Build: docker build -t model:v1.2
└─ Tag: model:v1.2-staging
└─ Push to registry (ECR/Harbor)
```



Step 3: Deploy to Staging

```
└─ Kubernetes namespace: staging
└─ Deployment: 2 replicas
└─ Resources:
    • CPU: 2 cores
    • Memory: 4Gi
    • GPU: Optional (1x T4)
└─ Service: ClusterIP
└─ Ingress: staging.irrigation.ma
```



Step 4: Integration Tests

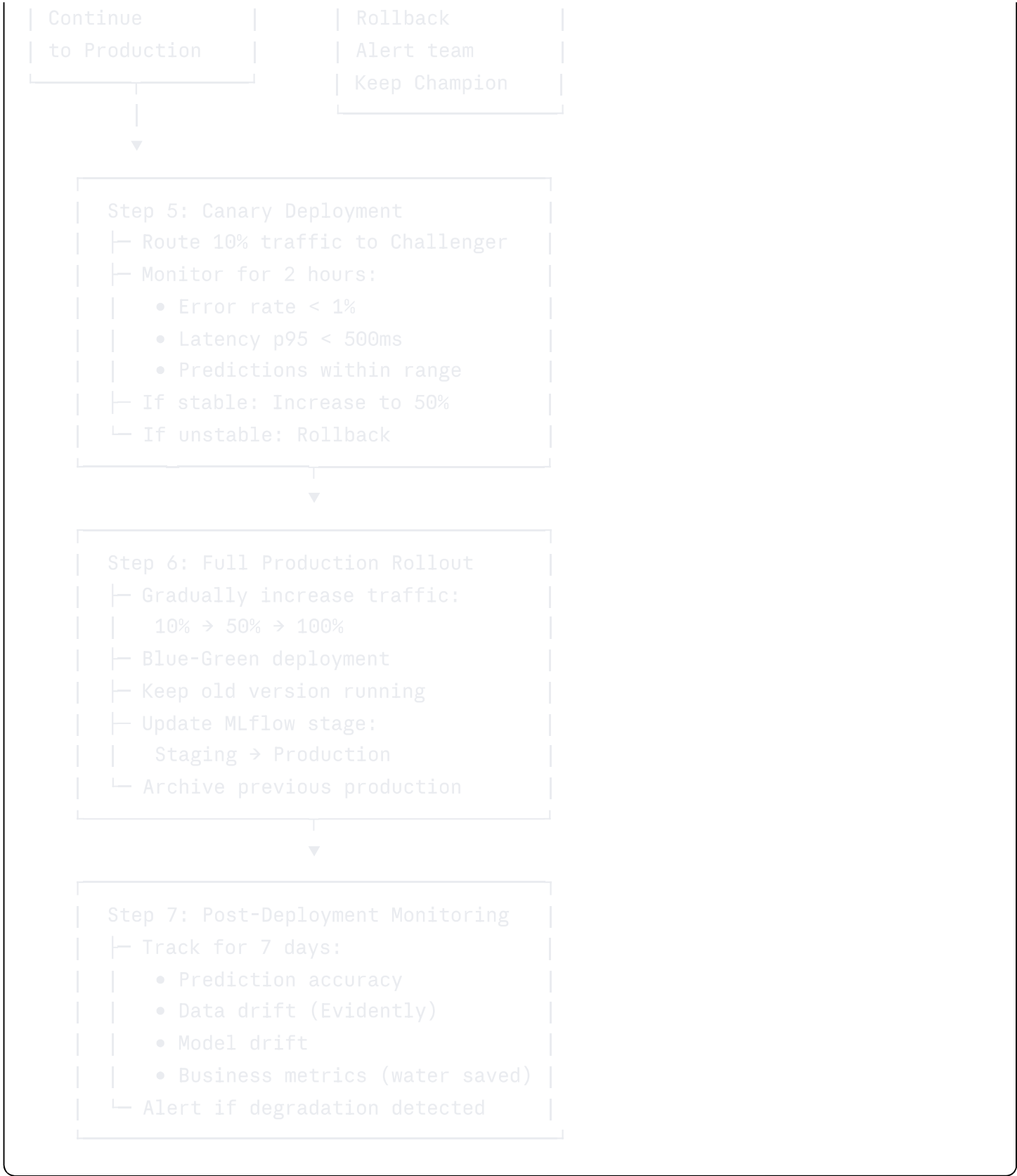
```
└─ Test 1: Health check
    GET /health → 200 OK
└─ Test 2: Prediction latency
    POST /predict → <500ms
└─ Test 3: Batch inference
    POST /batch → Process 100 items
└─ Test 4: Error handling
    Invalid input → 422 response
└─ Test 5: Load test (Locust)
    100 concurrent users → OK
```



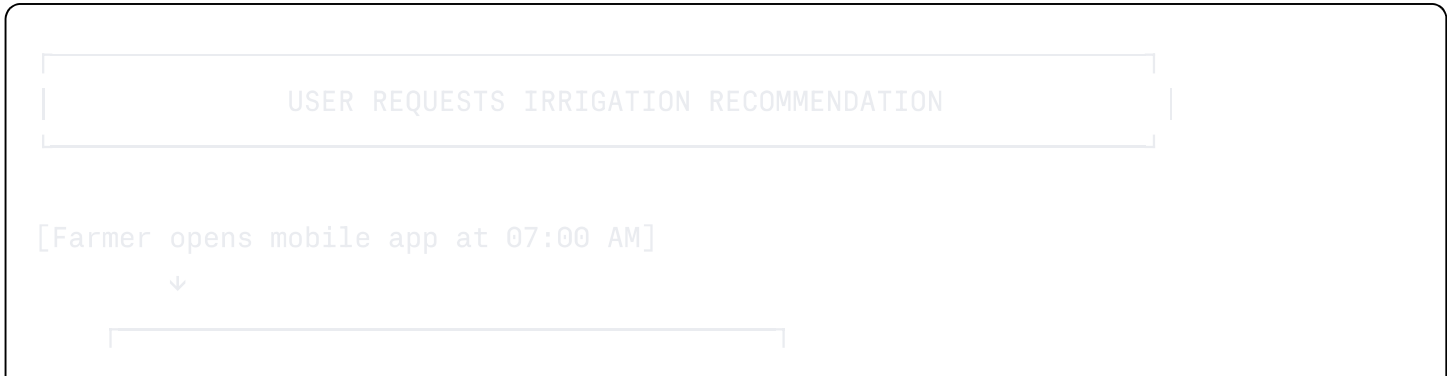
Tests Pass?

▼ YES

▼ NO



WORKFLOW 5: Real-time Inference & Recommendation



```
| Mobile App → API Gateway |
| POST /api/v1/predict/irrigation |
| { |
|   "field_id": "MA-CAS-F001", |
|   "crop_type": "wheat", |
|   "planting_date": "2024-11-15", |
|   "current_growth_stage": "heading" |
| } |
```



```
| FastAPI Service: Request Handler |
| └─ Authenticate user (JWT) |
| └─ Validate input (Pydantic) |
| └─ Extract field_id |
|   └─ Check Redis cache first |
|       Key: prediction:{field_id}:{date} |
```

▼ CACHE MISS

▼ CACHE HIT

```
| Proceed to |
| prediction |
```

```
| Return cached |
| result (50ms) |
```



```
| Feature Retrieval (Parallel) |
```



Weather (Redis)	Soil (Redis)	NDVI (Postgres /Redis)	DSSAT (Postgres /Redis)
• Temp	• Moist.	• Latest	• Growth
• Precip	• EC	index	stage
• ET0	• Temp	• Trend	• Kc
• Wind			• Stress



```
| Feature Assembly |
```

- └─ Combine all features into vector
- └─ Apply feature transformations
 - (scaling, encoding)
- └─ Create temporal sequences
 - (for LSTM: past 14 days)
- └─ Format for model input



Model Inference (Ensemble)

Model 1: Hybrid DSSAT-LSTM

- └─ Input: Feature vector + sequence
- └─ Forward pass
 - └─ Output: water_need_mm = 4.2

Model 2: Temporal Fusion Transformer

- └─ Input: Multi-horizon features
- └─ Attention weights computed
 - └─ Output: water_need_mm = 4.5

Model 3: XGBoost (Yield Impact)

- └─ Input: Current state + action
 - └─ Output: yield_impact_score

Ensemble Strategy: Weighted Average

- LSTM: 40% weight
- TFT: 40% weight
- DSSAT baseline: 20% weight

Final prediction: 4.35 mm



Recommendation Engine

- └─ Calculate irrigation schedule:
 - Amount: 43.5 m³/ha (4.35mm)
 - Timing: Today 18:00-20:00
 - Duration: 2.5 hours
 - Method: Drip irrigation
- └─ Confidence score: 87%
- └─ Reasoning (SHAP):
 - High ETO (4.8mm) → +1.2mm
 - Low soil moisture (45%) → +0.8
 - Heading stage (critical) → +0.5
 - No rain forecast (7d) → +0.3
- └─ Alternative scenarios:

```
| | • If irrigate tomorrow: -5% yield|
| | • If double amount: +0% yield |
| | • If skip: -12% yield |
| └─ Water savings vs traditional: |
|   35% less water, same yield |
└──────────────────────────────────┘
```



```
| Response Formatting |
| { |
|   "recommendation": { |
|     "irrigate": true, |
|     "amount_mm": 4.35, |
|     "amount_m3_per_ha": 43.5, |
|     "timing": "2024-05-16T18:00", |
|     "duration_hours": 2.5, |
|     "confidence": 0.87 |
|   }, |
|   "reasoning": { |
|     "top_factors": [ |
|       {"factor": "High ETO", |
|         "impact": "+1.2mm"}, |
|       {"factor": "Low soil moisture", |
|         "impact": "+0.8mm"} |
|     ] |
|   }, |
|   "forecast": { |
|     "next_7_days": [...] |
|   }, |
|   "alerts": [ |
|     "No rain forecasted for 7 days" |
|   ] |
| } |
└──────────────────────────────────┘
```



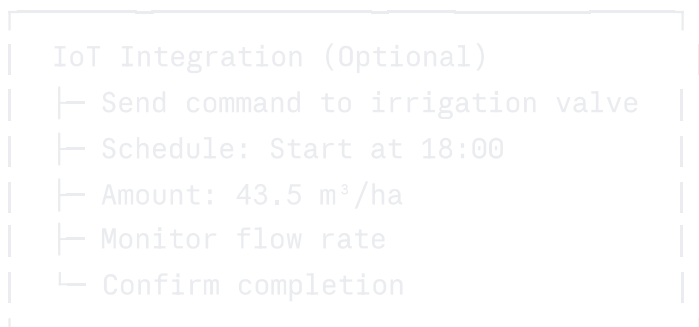
```
| Cache & Log |
| └─ Save to Redis (TTL: 1 hour) |
| └─ Log prediction to database |
|   (for model monitoring) |
| └─ Send to Kafka: prediction-events |
|   (for analytics) |
| └─ Update Prometheus metrics |
└──────────────────────────────────┘
```



```
| Return to Mobile App |
└──────────────────────────────────┘
```




[If farmer accepts recommendation]



Implémentation Phase par Phase (12 Semaines)

Phase 1: Infrastructure & Data Collection (Semaines 1-2)

Objectifs:

-  Infrastructure MLOps fonctionnelle
-  Pipelines d'ingestion opérationnels
-  Données historiques collectées

Tâches détaillées:

Semaine 1: Setup Infrastructure

bash

Jour 1-2: Configuration environnement

- └─ Install Docker, Kubernetes (minikube)
- └─ Setup GitHub repository
- └─ Create project structure
- └─ Configure CI/CD (GitHub Actions)
- └─ Deploy infrastructure (docker-compose.yml)
 - └─ Kafka + Zookeeper
 - └─ PostgreSQL + TimescaleDB
 - └─ Redis
 - └─ MinIO
 - └─ Airflow
 - └─ MLflow

Jour 3-4: API Access Setup

- └─ Register for APIs:
 - | └─ OpenWeatherMap (free tier)
 - | └─ NASA POWER
 - | └─ Sentinel Hub
 - | └─ Google Earth Engine (si disponible)
- └─ Test API connections
- └─ Implement rate limiting
- └─ Setup API key management

Jour 5: IoT Simulation (if no real sensors)

- └─ Create synthetic sensor data generator
- └─ MQTT broker setup (Mosquitto)
- └─ Simulate soil moisture, temperature
- └─ Test MQTT → Kafka bridge

Semaine 2: Data Collection

bash

```
Jour 1-2: Weather Data Pipeline
├─ Airflow DAG: weather_ingestion
├─ Fetch hourly weather data
├─ Validate and store in TimescaleDB
└─ Implement error handling

Jour 3-4: Satellite Data Pipeline
├─ Download Sentinel-2 historical data
│   Région: Votre zone d'étude au Maroc
│   Période: 2 dernières années
├─ Preprocessing pipeline:
│   ├─ Cloud masking
│   ├─ Atmospheric correction
│   └─ Calculate NDVI, EVI, LAI
└─ Store in PostGIS

Jour 5: Historical Data Collection
├─ Collect:
│   ├─ Historical yield data
│   ├─ Historical irrigation records
│   ├─ Soil properties
│   └─ Crop calendar
├─ Data quality checks
└─ Exploratory Data Analysis (EDA)
```

Livrables:

- ✓ Infrastructure running on localhost
- ✓ 90 days of weather data collected
- ✓ 24 months of satellite imagery
- ✓ EDA notebook with data insights

Phase 2: DSSAT Integration (Semaines 3-4)

Objectifs:

- ✓ DSSAT installé et configuré
- ✓ Modèle calibré pour votre région
- ✓ Simulations de base fonctionnelles

Semaine 3: DSSAT Setup & Calibration

```
bash
```

Jour 1: DSSAT Installation

- └─ Install DSSAT v4.8
- └─ Setup Python interface (dssat-csm-python)
- └─ Test basic simulation
- └─ Understand file formats (.WTH, .SOL, .MZX)

Jour 2-3: Soil Data Preparation

- └─ Collect soil data for your region:
 - | └─ Texture (sand, silt, clay %)
 - | └─ Bulk density
 - | └─ Organic carbon
 - | └─ pH, CEC
- └─ Create .SOL file
- └─ Validate with field measurements

Jour 4-5: Crop Calibration

- └─ Select crop (ex: Blé dur)
- └─ Gather phenology data:
 - | └─ Planting to emergence (days)
 - | └─ Emergence to flowering
 - | └─ Flowering to maturity
 - | └─ Thermal time requirements
- └─ Calibrate genetic coefficients
- └─ Run sensitivity analysis
- └─ Validate with observed yields

Semaine 4: DSSAT Automation

python

Exemple: Automated DSSAT run

```
from dssat import DSSAT
```

```
import pandas as pd
```

```
def run_dssat_simulation(field_id, weather_data, soil_params):
```

```
    """
```

```
    Run DSSAT simulation for irrigation optimization
```

```
    """
```

```
    # Initialize DSSAT
```

```
    dssat = DSSAT()
```

```
    # Prepare input files
```

```
    weather_file = create_weather_file(weather_data)
```

```
    soil_file = create_soil_file(soil_params)
```

```
    crop_file = create_crop_management_file(field_id)
```

```
    # Run simulation
```

```
    results = dssat.run(
```

```
        weather=weather_file,
```

```
        soil=soil_file,
```

```
        management=crop_file,
```

```
        cultivar='IB0488' # Exemple: Moroccan wheat cultivar
```

```
)
```

```
    # Extract key outputs
```

```
    outputs = {
```

```
        'water_stress': results['SWFAC'], # Soil water factor
```

```
        'crop_coefficient': results['Kc'],
```

```
        'biomass': results['CWAD'],
```

```
        'yield': results['HWAH'],
```

```
        'et_actual': results['ETAA']
```

```
    }
```

```
    return outputs
```

Airflow DAG implementation

Jour 1-2: Airflow DAG for DSSAT

└ DAG: dssat_weekly_simulation

└ Task 1: Fetch latest weather

└ Task 2: Prepare DSSAT inputs

└ Task 3: Run simulations for all fields

└ Task 4: Extract and store outputs

└ Task 5: Validate results

Jour 3-4: Scenario Analysis

└ Current irrigation scenario

- └ Deficit irrigation (70%, 80%, 90%)
- └ Climate change scenarios (+2°C, -20% rain)
- └ Compare water use efficiency
- └ Identify optimal strategies

Jour 5: Integration Tests

- └ Test end-to-end DSSAT pipeline
- └ Validate outputs against field data
- └ Calculate error metrics (RMSE, NRMSE)
- └ Document calibration parameters

Livrables:

- ☒ Calibrated DSSAT model ($R^2 > 0.80$ for yield)
- ☒ Automated weekly simulation pipeline
- ☒ Baseline water stress predictions

Phase 3: Feature Engineering (Semaine 5)

Objectifs:

- ☒ Feature store opérationnel
- ☒ Features ML prêtes pour l'entraînement

Code complet en un exemple:

```
python
```

```
# src/features/feature_engineering.py
import pandas as pd
import numpy as np
from feast import FeatureStore
from datetime import datetime, timedelta

class IrrigationFeatureEngine:
    def __init__(self):
        self.store = FeatureStore(repo_path="feast_repo/")

    def create_temporal_features(self, df):
        """Create time-based features"""
        df = df.copy()

        # Rolling aggregations
        df['temp_rolling_7d'] = df['temperature'].rolling(7).mean()
        df['precip_rolling_7d'] = df['precipitation'].rolling(7).sum()
        df['et0_cumulative_30d'] = df['et0'].rolling(30).sum()

        # Growing Degree Days (GDD)
        df['gdd'] = np.maximum(df['temperature'] - 10, 0) # Base temp 10°C
        df['gdd_cumulative'] = df['gdd'].cumsum()

        # Days since last irrigation
        last_irrigation = df[df['irrigation'] > 0].index
        df['days_since_irrigation'] = (df.index - last_irrigation[-1]).days

        return df

    def create_crop_features(self, df, ndvi_data):
        """Create crop health features"""
        df = df.copy()

        # NDVI trend
        df['ndvi'] = ndvi_data
        df['ndvi_trend_7d'] = df['ndvi'].diff(7)
        df['ndvi_trend_14d'] = df['ndvi'].diff(14)

        # Crop coefficient (from DSSAT)
        df['kc'] = self.get_kc_from_growth_stage(df['growth_stage'])

        # Water stress index
        df['water_stress_index'] = 1 - (df['soil_moisture'] / df['field_capacity'])

        return df
```

```

def create_climate_features(self, df):
    """Create climate anomaly features"""
    df = df.copy()

    # Temperature anomalies (vs historical average)
    historical_avg = self.get_historical_climate()
    df['temp_anomaly'] = df['temperature'] - historical_avg['temperature']
    df['precip_anomaly'] = df['precipitation'] - historical_avg['precipitation']

    # Extreme event flags
    df['extreme_heat'] = (df['temperature'] > df['temperature'].quantile(0.95)).
    df['drought_days'] = (df['precipitation'] == 0).rolling(14).sum()

    return df

def create_soil_water_balance(self, df):
    """Calculate soil water balance"""
    df = df.copy()

    # Water balance equation
    #  $SW(t) = SW(t-1) + Precip + Irrigation - ET - Drainage - Runoff$ 
    df['soil_water'] = (
        df['soil_water'].shift(1) +
        df['precipitation'] +
        df['irrigation'] -
        df['et_actual'] -
        df['deep_drainage'] -
        df['runoff']
    )

    # Depletion fraction
    df['depletion_fraction'] = 1 - (df['soil_water'] / df['total_available_water'])

    # Readily available water (RAW)
    df['raw_depleted'] = (df['depletion_fraction'] > 0.5).astype(int)

    return df

def engineer_all_features(self, field_id, start_date, end_date):
    """Main feature engineering pipeline"""

    # Load raw data
    weather = self.load_weather_data(field_id, start_date, end_date)
    soil = self.load_soil_data(field_id, start_date, end_date)
    ndvi = self.load_satellite_data(field_id, start_date, end_date)
    dssat = self.load_dssat_outputs(field_id, start_date, end_date)
    irrigation = self.load_irrigation_history(field_id, start_date, end_date)

```



```

# Merge all data
df = pd.merge(weather, soil, on='date', how='left')
df = pd.merge(df, irrigation, on='date', how='left')

# Create features
df = self.create_temporal_features(df)
df = self.create_crop_features(df, ndvi)
df = self.create_climate_features(df)
df = self.create_soil_water_balance(df)

# Add DSSAT predictions as features
df = pd.merge(df, dssat[['date', 'predicted_stress', 'predicted_kc']],
              on='date', how='left')

# Create target variable (water need for next day)
df['target_water_need'] = df['et_actual'].shift(-1) # Tomorrow's ET

# Drop rows with missing targets
df = df.dropna(subset=['target_water_need'])

# Save to Feature Store
self.save_to_feature_store(df, field_id)

return df

```

```

# Feast Feature Store definition
# feast_repo/features.py
from feast import Entity, FeatureView, Field, FileSource
from feast.types import Float32, Int64, String
from datetime import timedelta

```

```

# Define entity
field_entity = Entity(
    name="field_id",
    description="Agricultural field identifier"
)

```

```

# Weather features
weather_features = FeatureView(
    name="weather_features",
    entities=[field_entity],
    ttl=timedelta(days=1),
    schema=[
        Field(name="temperature", dtype=Float32),
        Field(name="temp_rolling_7d", dtype=Float32),
        Field(name="precipitation", dtype=Float32),

```

```

        Field(name="precip_rolling_7d", dtype=Float32),
        Field(name="et0", dtype=Float32),
        Field(name="et0_cumulative_30d", dtype=Float32),
        Field(name="gdd_cumulative", dtype=Float32),
    ],
    online=True,
    source=FileSource(
        path="data/features/weather.parquet",
        timestamp_field="timestamp"
    )
)

# Crop features
crop_features = FeatureView(
    name="crop_features",
    entities=[field_entity],
    ttl=timedelta(days=1),
    schema=[
        Field(name="ndvi", dtype=Float32),
        Field(name="ndvi_trend_7d", dtype=Float32),
        Field(name="kc", dtype=Float32),
        Field(name="water_stress_index", dtype=Float32),
        Field(name="growth_stage", dtype=String),
    ],
    online=True,
    source=FileSource(
        path="data/features/crop.parquet",
        timestamp_field="timestamp"
    )
)

```

Livrables Semaine 5:

- ☒ 50+ features engineered
- ☒ Feature store configuré
- ☒ Documentation de chaque feature

Je vais continuer avec les phases suivantes si vous le souhaitez. Voulez-vous que je poursuive avec:

- Phase 4: Model Training (Semaines 6-7)
- Phase 5: Model Deployment (Semaines 8-9)
- Phase 6: Monitoring & Production (Semaines 10-12)

Ou préférez-vous que je détaille un aspect spécifique du projet?